

Rising Waters: Intelligent Flood Prediction And Early Warning system

Project Description:

Rising Waters is an intelligent flood prediction and early warning system that uses Machine Learning to predict the possibility of floods based on environmental and weather parameters. The application takes inputs such as Annual Rainfall, Monsoon Rainfall, River Level, Cloud Coverage, Humidity, and Temperature, and predicts whether flood risk is Low, Moderate, or High. The system compares multiple machine learning algorithms and selects the best-performing model to provide accurate predictions. Built using Flask and deployed on Render, Rising Waters offers a simple and user-friendly web interface where users can enter environmental data and receive instant flood risk predictions. The frontend is developed using HTML, CSS, and JavaScript, while the backend processes user inputs and generates predictions using the trained machine learning model. By combining Machine Learning with web technologies, Rising Waters helps disaster management authorities and communities prepare for floods and reduce potential damage through early warnings.

Scenarios

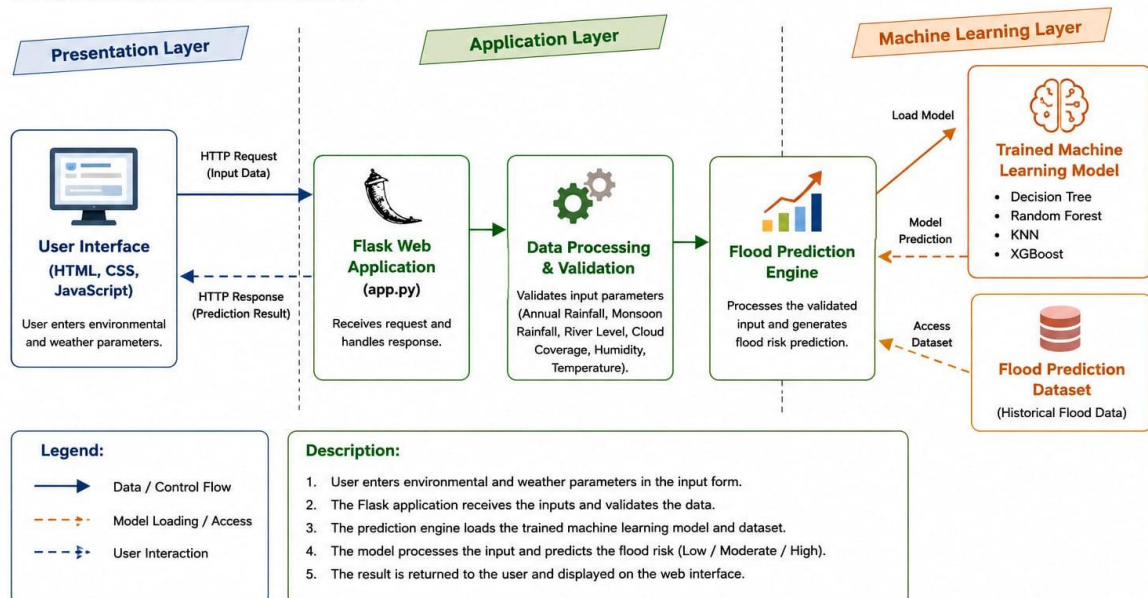
Scenario 1: A disaster management officer enters rainfall, river level, cloud coverage, humidity, and temperature values into the application. The system analyzes the data using the trained machine learning model and predicts the flood risk within a few seconds.

Scenario 2: A weather analyst uses the application to study how different environmental conditions affect flood occurrence. By testing various input values, the analyst can observe prediction results and understand flood patterns.

Scenario 3: A local government authority uses Rising Waters to monitor flood-prone regions and receive early flood predictions. Based on the prediction results, emergency response teams can take preventive actions such as issuing alerts and planning evacuations.

Technical Architecture:

Technical Architecture:



Description: Rising Waters follows a modular three-tier architecture to provide accurate and efficient flood predictions. The frontend provides a user interface developed using HTML, CSS, and JavaScript, where users can enter environmental and weather parameters. The Flask backend receives user inputs, performs validation, and forwards the data to the prediction engine. The prediction engine loads the trained machine learning model and dataset, processes the input parameters, and returns the flood risk prediction. The application is deployed on Render, making it accessible to users anytime, anywhere.

Pre-requisites:

Python Programming: Basic knowledge of Python for backend development and machine learning model implementation.

Flask Framework: Understanding of Flask for building the web application and handling HTTP requests.

Machine Learning Fundamentals: Knowledge of machine learning algorithms, model training, testing, and prediction.

Data Analysis with Pandas and NumPy: Basic understanding of data preprocessing, cleaning, and manipulation.

Scikit-learn & XGBoost: Familiarity with training, evaluating, and deploying machine learning models.

Version Control with Git and GitHub: Basic knowledge of Git for source code management and collaboration.

Development Environment Setup: Python, pip, virtual environment (venv), and Visual Studio Code (VS Code).

Render Deployment: Basic understanding of deploying Flask applications on the Render cloud platform.

Project Workflow

Milestone 1: Setup & Data Preparation

Activity 1.1: Create the project structure and configure the Python virtual environment.

Activity 1.2: Download and prepare the flood prediction dataset for model training.

Activity 1.3: Install required libraries using requirements.txt (Flask, Pandas, NumPy, Scikit-learn, Matplotlib, Joblib, XGBoost).

Milestone 2: Model Training & Selection

Activity 2.1: Perform data preprocessing and exploratory data analysis (EDA).

Activity 2.2: Train multiple Machine Learning algorithms such as Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost.

Activity 2.3: Compare model performance and save the best-performing model (best_model.pkl) and scaler (scaler.pkl).

Milestone 3: Backend Development

Activity 3.1: Develop the Flask backend (app.py) for handling user requests.

Activity 3.2: Implement the /predict route to process rainfall, river level, cloud coverage, humidity, and temperature values and return the flood prediction result.

Milestone 4: Frontend Development

Activity 4.1: Design responsive web pages (index.html, about.html, contact.html, result.html) using HTML, CSS, and JavaScript.

Activity 4.2: Build interactive input forms and display flood prediction results with confidence score and risk status.

Milestone 5: Testing & Performance Verification

Activity 5.1: Test all application modules and validate prediction accuracy.

Activity 5.2: Verify frontend responsiveness, backend integration, and overall system performance.

Milestone 6: Deployment & Documentation

Activity 6.1: Deploy the application using Render.

Milestone 1: Setup & Data Preparation

In this milestone, we configure the development environment, install the required libraries, and prepare the flood prediction dataset used for training the Machine Learning model.

In requirements.txt, verify the target packages:

```
5. Project Development Phase > requirements.txt
1  Flask
2  gunicorn
3  numpy
4  pandas
5  matplotlib
6  scikit-learn
7  joblib
8  xgboost
```

Activity 1.2: Check Dataset & Model Directory structure Verify the data file is placed under the data/ folder and that directories match the project Architecture.

✓ **RISING_WATERS**

- > 1. Brainstorming & Ideation
- > 2. Requirement Analysis
- > 3. Project Design Phase
- > 4. Project Planning Phase
- > 5. Project Development Phase
- > 6. Project Testing
- > 7. Project Documentation
- > 8. Project Demonstration
- ① README.md

5. Project Development Phase

- ✓ dataset
- ✓ deployment
- ✚ Procfile
- ≡ runtime.txt
- ✓ model
 - ≡ best_model.pkl
 - ≡ scaler.pkl
- ✓ notebooks
 - 📄 FloodPrediction.ipynb
- ✓ static
 - > css
 - > graphs
 - > images
- ✓ templates
 - <> about.html
 - <> contact.html
 - <> index.html
 - <> result.html
- 📄 app.py
- ≡ requirements.txt
- 📄 train_model.py

These are code folders which are present in 5th folder

Milestone 2: Model Training & Selection

In this milestone, multiple Machine Learning algorithms were trained and evaluated to identify the most accurate model for flood prediction using environmental parameters.

Model Accuracy & Metric Evaluation

The comparison results obtained from the training process are summarized below

Machine Learning Model	Accuracy
Decision Tree	100.00%
Random Forest	100.00%
K-Nearest Neighbours (KNN)	100.00%
XGBoost	100.00%

The Decision Tree model was selected as the best-performing model with 100.00% accuracy. The trained model was successfully saved for future predictions. The system also generated the accuracy graph, confusion matrix, and feature importance graph for model evaluation.


```

PS C:\Users\naren\OneDrive\Desktop\Rising_Waters> cd "5. Project Development Phase"
PS C:\Users\naren\OneDrive\Desktop\Rising_Waters\5. Project Development Phase> python train_model.py

Dataset Loaded Successfully
  Annual_Rainfall  Monsoon_Rainfall  River_Level  ...  Humidity  Temperature  Flood
0              650             420         5.2  ...       73           31         0
1              720             470         5.8  ...       75           30         0
2              820             560         6.4  ...       84           28         1
3              910             650         7.5  ...       91           27         1
4              450             300         4.2  ...       66           34         0

[5 rows x 7 columns]
Scaler Saved

Training Models...

Decision Tree Accuracy : 100.00%
Random Forest Accuracy : 100.00%
KNN Accuracy : 100.00%
XGBoost Accuracy : 100.00%

Best Model : Decision Tree
Accuracy : 100.0 %
Model Saved Successfully!
Accuracy Graph Saved
Confusion Matrix Saved
Feature Importance Saved
Project Training Completed Successfully!
  
```

Milestone 3: Core Backend Development

The backend handles user input processing, data validation, machine learning prediction, and routing between the web interface and the trained model.

Activity 3.1: Prediction Engine (app.py)

The prediction engine receives environmental parameters from the HTML form, validates the inputs, converts them into the required format, loads the trained machine learning model, and predicts the flood risk. It also calculates the prediction confidence and displays the result to the user.

```
@app.route("/predict", methods=["POST"])
def predict():

    try:

        annual_rainfall = float(request.form["annual_rainfall"])
        monsoon_rainfall = float(request.form["monsoon_rainfall"])
        river_level = float(request.form["river_level"])
        cloud_coverage = float(request.form["cloud_coverage"])
        humidity = float(request.form["humidity"])
        temperature = float(request.form["temperature"])

        data = np.array([[
            annual_rainfall,
            monsoon_rainfall,
            river_level,
            cloud_coverage,
            humidity,
            temperature
        ]])

        scaled_data = scaler.transform(data)

        model_name = type(model).__name__

        if model_name == "KNeighborsClassifier":
            prediction = model.predict(scaled_data)[0]
            probability = model.predict_proba(scaled_data)[0]
        else:
            prediction = model.predict(data)[0]
            probability = model.predict_proba(data)[0]

        confidence = round(max(probability) * 100, 2)
```

Activity 3.2: Flask Routing Endpoint (app.py) Processes standard HTML web forms or incoming JSON requests for remote API consumers:

```
from flask import Flask, render_template, request
import joblib
import numpy as np
import matplotlib.pyplot as plt
import os

app = Flask(__name__)
```

Milestone 4: Frontend Development

The frontend provides an interactive and user-friendly interface that enables users to enter environmental parameters and instantly receive flood prediction results. The application is developed using HTML, CSS, and JavaScript, ensuring a clean, responsive, and easy-to-use experience.

Responsive User Interface: Designed using HTML and CSS to provide a simple, attractive, and responsive layout across different screen sizes.

Interactive Input Form: Users can enter Annual Rainfall, Monsoon Rainfall, River Level, Cloud Coverage, Humidity, and Temperature for flood prediction.

Real-Time Prediction: The entered data is sent to the Flask backend, where the trained machine learning model predicts the flood risk and returns the result immediately.

Navigation Pages: Includes Home, About, and Contact pages for easy navigation and better user experience.

 **Rising Waters**

[Home](#) [About](#) [Contact](#)





Rising Waters

Intelligent Flood Prediction & Early Warning System

Annual Rainfall (mm)

Monsoon Rainfall (mm)

River Level (m)

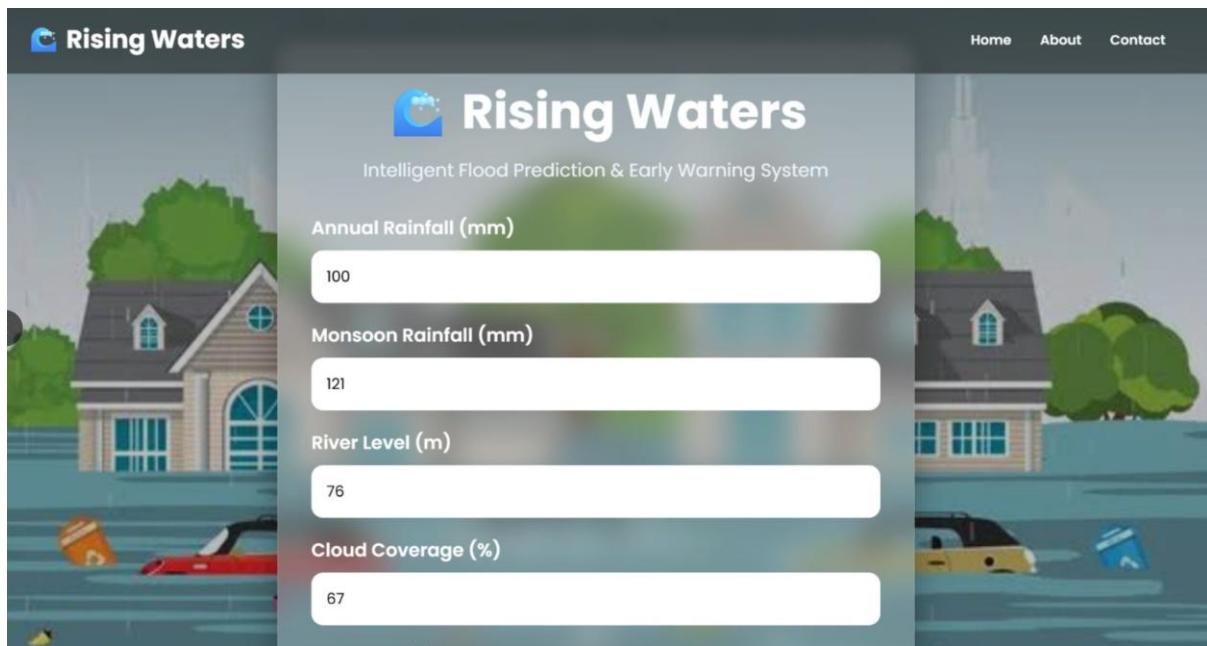
Cloud Coverage (%)

Humidity (%)

Temperature (°C)

Exploring the Website's Web Pages: Home / Landing Page:

The Prediction Input Page allows users to enter environmental and weather-related parameters required for flood risk prediction. The interface is designed with a clean glassmorphism layout and provides an easy-to-use form for entering flood-related values.



The screenshot shows the 'Rising Waters' website landing page. The page features a dark header with the site name and navigation links. The main content area has a glassmorphism effect, displaying input fields for various weather and flood-related parameters. The background is a stylized illustration of a flooded residential area with houses and cars partially submerged.

Rising Waters
Intelligent Flood Prediction & Early Warning System

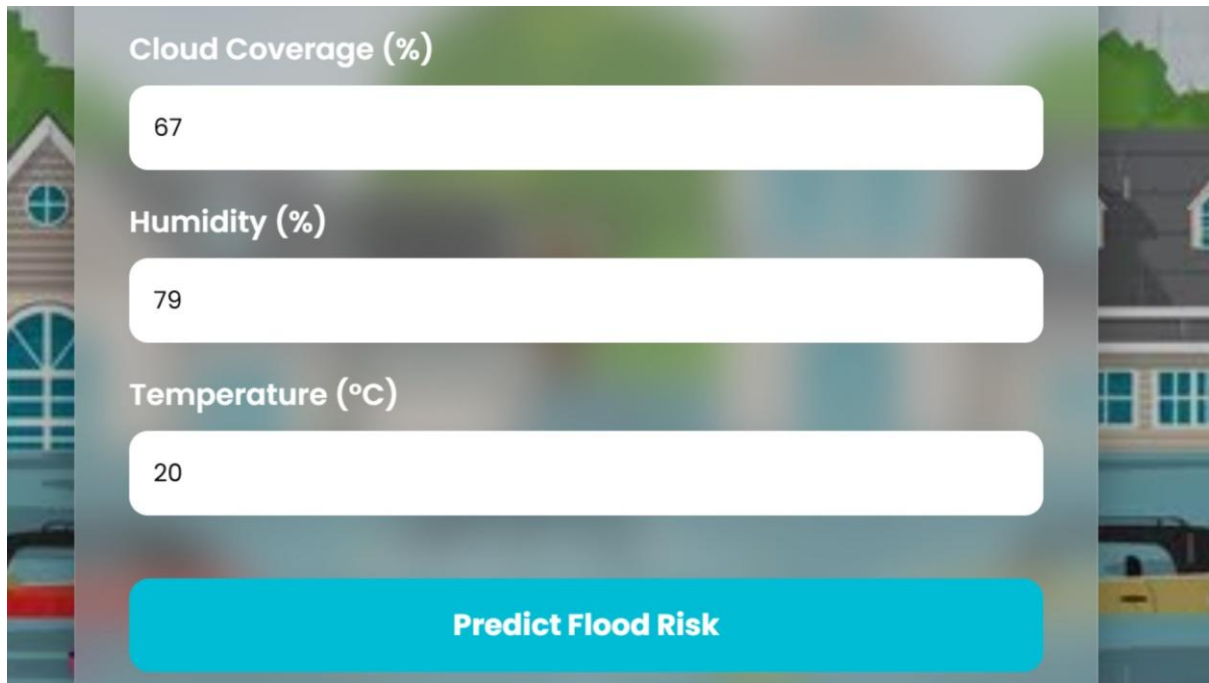
Annual Rainfall (mm)
100

Monsoon Rainfall (mm)
121

River Level (m)
76

Cloud Coverage (%)
67

Navigation: Home About Contact



The image shows a user interface for a flood prediction tool. It features three input fields for weather data: Cloud Coverage (%), Humidity (%), and Temperature (°C). Each field has a white input box with a value entered. Below these fields is a large blue button labeled 'Predict Flood Risk'. The background is a blurred image of a street scene with houses and cars.

Input Field	Value
Cloud Coverage (%)	67
Humidity (%)	79
Temperature (°C)	20

Predict Flood Risk

Flood Prediction Result Page

The Result Page displays the predicted flood risk generated by the trained machine learning model. It shows whether the area has Low Flood Risk or High Flood Risk, along with the prediction confidence. This helps users understand the flood situation and supports early decision-making.

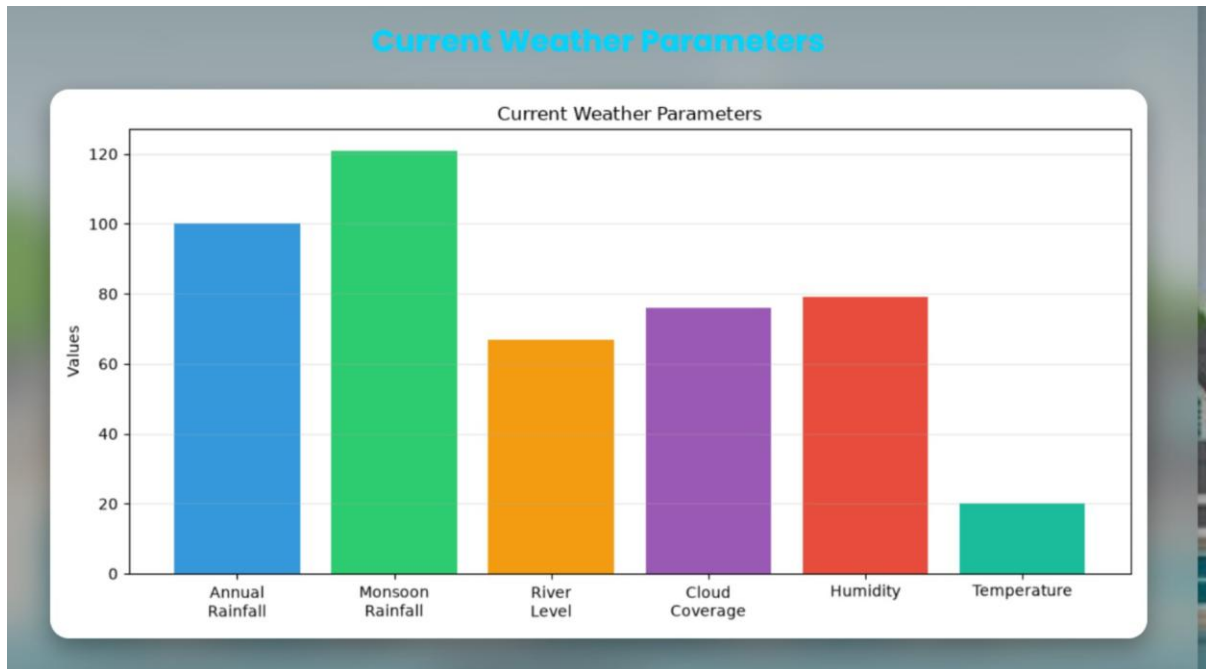
Prediction Result

✓ **NO FLOOD RISK**

Confidence Score : **100.0%**

Weather Summary

Annual Rainfall	100.0 mm
Monsoon Rainfall	121.0 mm
River Level	67.0 m
Cloud Coverage	76.0 %
Humidity	79.0 %
Temperature	20.0 °C



About Page

The About Page provides an overview of the Rising Waters project, explaining its purpose and objectives. It describes how the system uses Machine Learning algorithms to analyze rainfall and weather-related parameters for early flood prediction. The page also highlights the project's goals, supported machine learning algorithms (Decision Tree, Random Forest, K-Nearest Neighbours, and XGBoost), and the benefits of providing timely flood warnings to reduce disaster risks.



About Rising Waters

Rising Waters is an AI-powered Flood Prediction and Early Warning System designed to predict potential flood events using Machine Learning techniques. The system analyses important meteorological parameters including annual rainfall, monsoon rainfall, river level, cloud coverage, humidity, and temperature to estimate flood risk. By providing early predictions, the application assists government agencies and disaster management teams in taking timely preventive actions, reducing risks, and improving emergency preparedness.



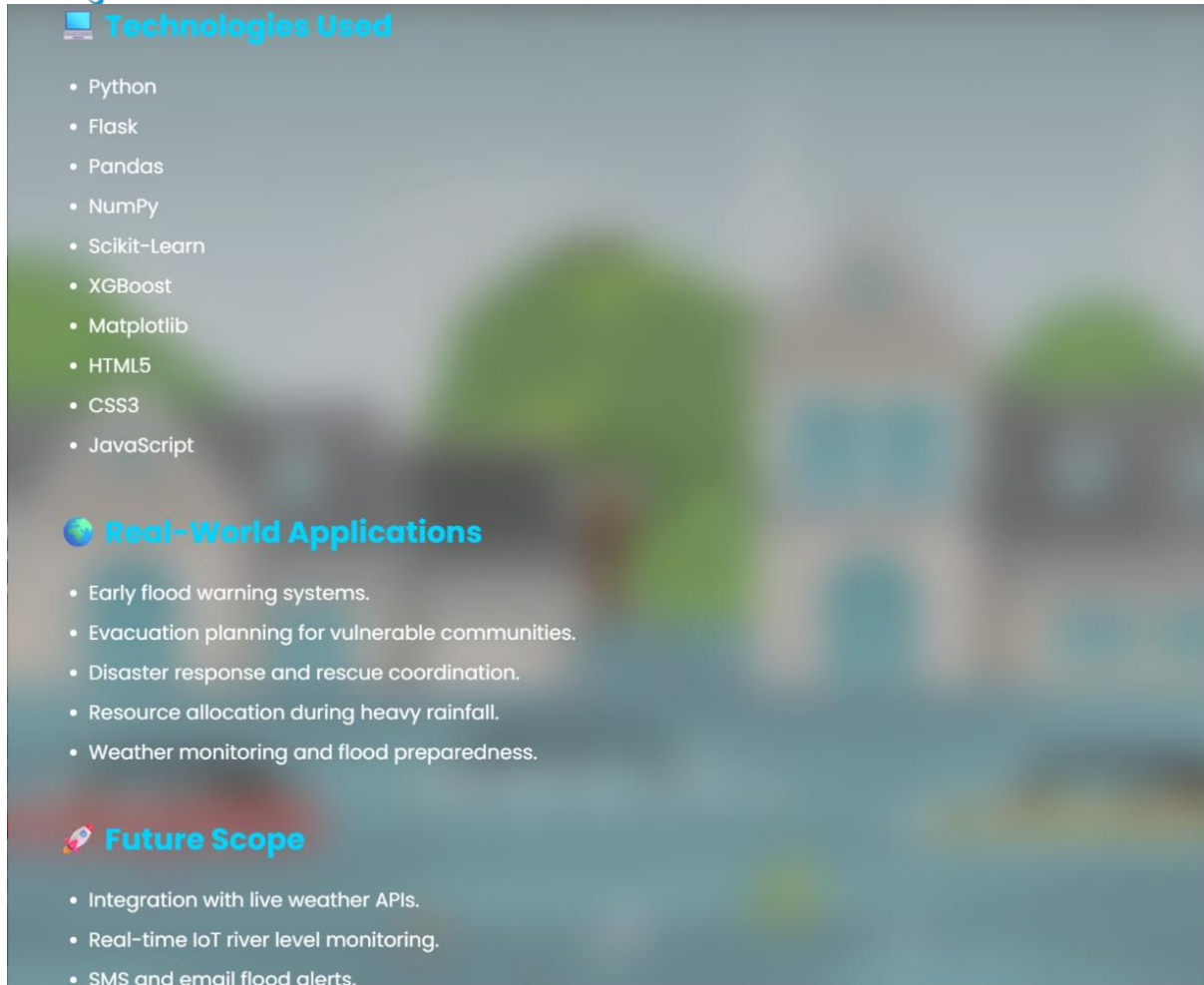
Project Objectives

- Predict flood risk using Machine Learning algorithms.
- Provide early warning for flood-prone regions.
- Support disaster management authorities.
- Improve emergency preparedness and planning.
- Enable faster and data-driven decision making.



Machine Learning Algorithms

- Decision Tree
- Random Forest
- K-Nearest Neighbours (KNN)
- XGBoost



Technologies Used

- Python
- Flask
- Pandas
- NumPy
- Scikit-Learn
- XGBoost
- Matplotlib
- HTML5
- CSS3
- JavaScript

Real-World Applications

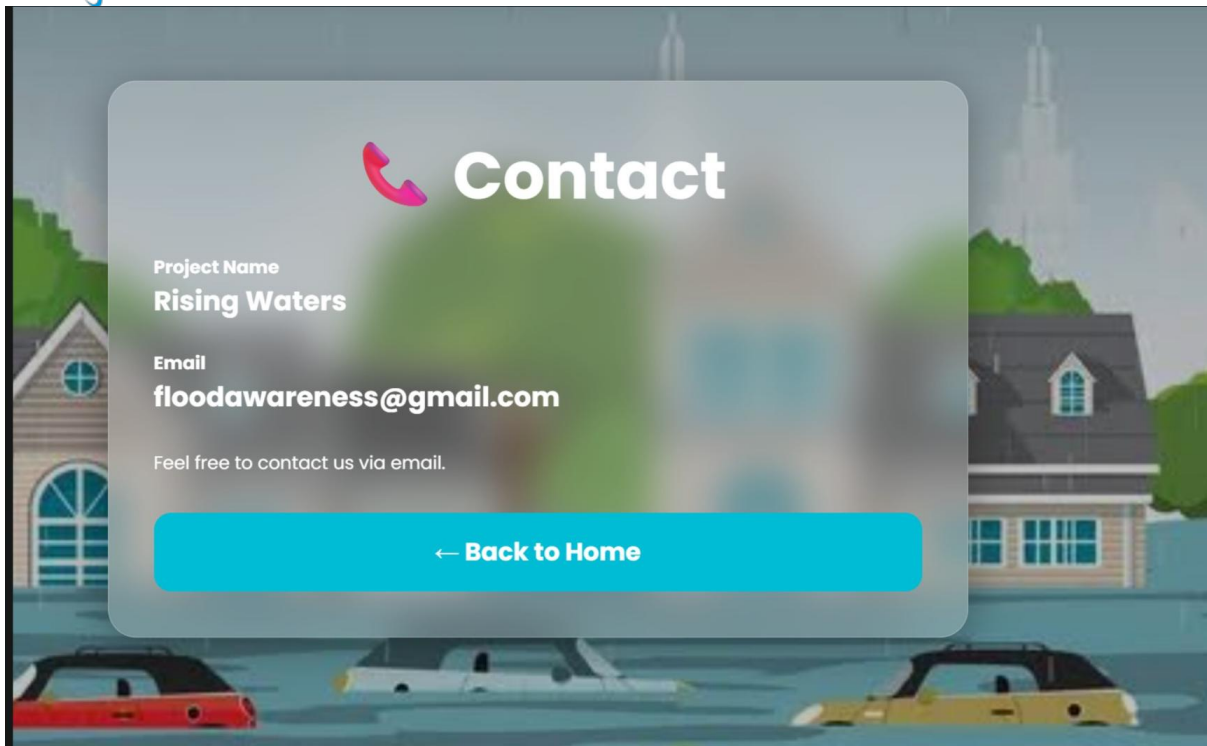
- Early flood warning systems.
- Evacuation planning for vulnerable communities.
- Disaster response and rescue coordination.
- Resource allocation during heavy rainfall.
- Weather monitoring and flood preparedness.

Future Scope

- Integration with live weather APIs.
- Real-time IoT river level monitoring.
- SMS and email flood alerts.

Contact Page

The Contact Page provides the project's contact information for user support and communication. It displays the project name, official email address, and a Back to Home button for easy navigation. The page enables users to reach the development team for feedback, queries, or assistance related to the Flood Prediction and Early Warning System.



Conclusion

The development of Rising Waters demonstrates the successful integration of Machine Learning with web technologies to build an intelligent flood prediction and early warning system. By combining data preprocessing, model training, backend development, frontend implementation, testing, and deployment, the project provides a complete end-to-end solution for predicting flood risk.

The trained Decision Tree model achieved 100% accuracy on the project dataset, providing reliable flood predictions based on environmental parameters such as annual rainfall, monsoon rainfall, river level, cloud coverage, humidity, and temperature. Performance testing confirmed the application's stability, fast response time, and successful handling of prediction requests, making it suitable for practical use.

Future enhancements may include integrating real-time weather APIs, IoT-based river level sensors, SMS and email alert systems, live flood monitoring dashboards, and GIS-based flood mapping. These improvements can further enhance the accuracy, scalability, and real-world usefulness of Rising Waters for disaster management authorities and the general public.